Experiment 8: Depth First Search (DFS)

Aim

To write a Python program to implement Depth First Search (DFS) traversal on a graph.

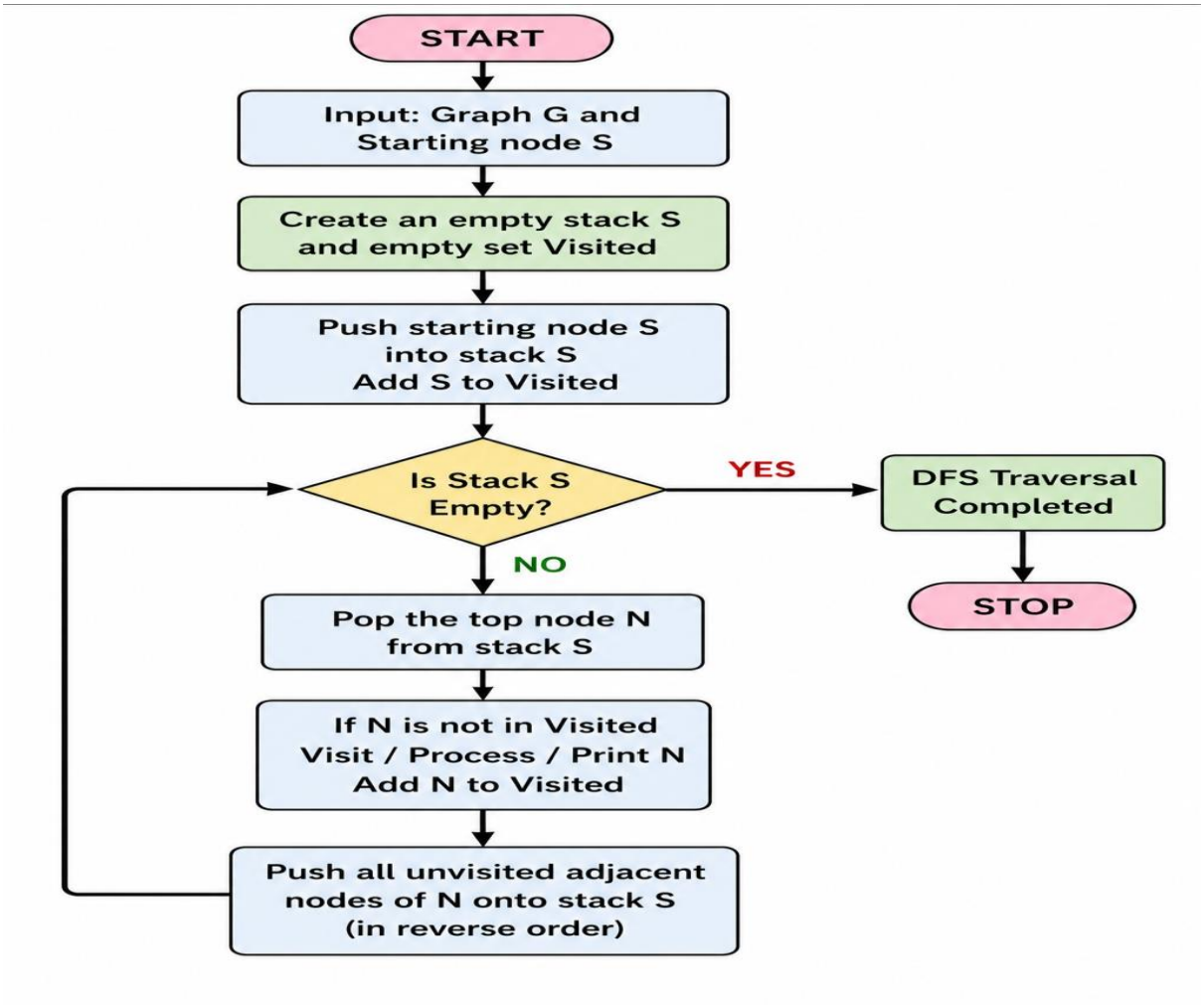
Objective

To traverse all the nodes of a graph using the DFS algorithm and understand the working of stack/recursion-based graph traversal.

Algorithm

- Create a graph using an adjacency list.
- Define a DFS function with the current node as input.
- Mark the current node as visited.
- Print the current node.
- Recursively visit all unvisited adjacent nodes.
- Repeat until all reachable nodes are visited.

Flowchart:



Code

Python

Run

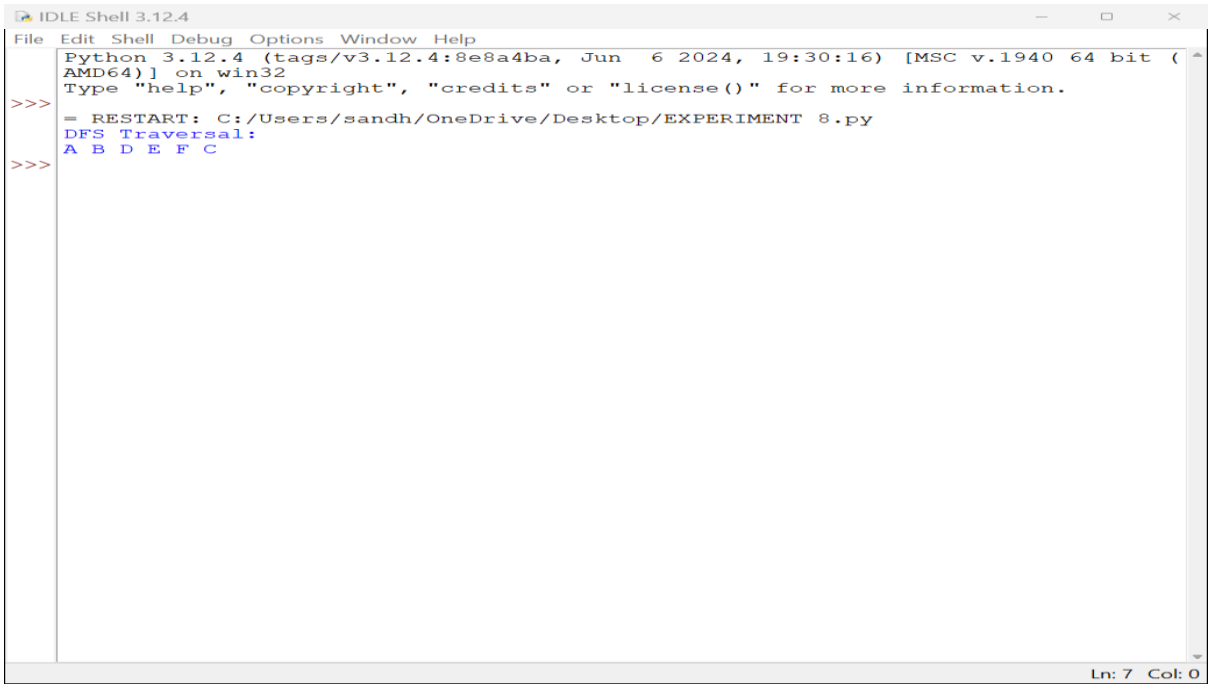
```
g = {'A':['B','C'], 'B':['D'], 'C':['E'], 'D':[], 'E':[]}


v = set()


def dfs(n):
    if n not in v:
        print(n, end=' ')
        v.add(n)
        for i in g[n]:
            dfs(i)


print("DFS Traversal:")
dfs('A')
```

Output



Result

The DFS traversal of the given graph was successfully performed using Python, and the nodes were visited in the order: **A → B → D → C → E**.

Conclusion

The Depth First Search algorithm was implemented successfully in Python. DFS explores nodes deeply before backtracking and is useful for path finding, cycle detection, and graph-based applications.